

CanvaGrid

-App Assistant for Image Transfer on Canvas-

Anthopoulou Elpida-Kalliopi

AEM 132

Interactive Applications for Mobile Devices

Technologies of Interactive Systems

Aristotle University of Thessaloniki

June 2026

Table of Contents

Executive Summary.....	2
Part 1: Purpose and Target Users.....	3
1.1 Purpose of the Application.....	3
1.2 Target Users.....	3
Part 2: Innovation and Differentiation.....	4
Part 3: User Scenarios and Functionality.....	5
3.1 User Scenario I - The Painter.....	5
3.2 User Scenario II - The Art Student.....	6
3.3 User Scenario III - The Hobbyist.....	8
Part 4: Source Code Explanation.....	11
4.1 Database Design and Architecture.....	11
4.2 Main Hub Navigation and Saved Projects Management.....	12
4.3 EditorActivity Workspace and Grid Rendering.....	14
4.3.1 EditorActivity Workspace Management.....	14
4.3.2 GridView Overlay Rendering.....	14
4.4 SuggestSizeActivity and Canvas Recommendation Algorithm.....	15
4.5 Help System and Dialog Lifecycle Management.....	16
4.6 Scoped Storage Architecture and Data Privacy Management.....	16
Part 5: Future Additions.....	18
5.1 Potential AI Integration and Tablet Adaptations.....	18
5.2 Future AR Features and Cloud Syncing.....	18

Executive Summary

The transition from a digital reference image to a physical canvas presents a common challenge for artists, as manually transferring proportions and composition can lead to inaccuracies, distortion, or unwanted cropping. CanvaGrid modernizes the traditional grid method through a native Android application designed to simplify the preparation stage of the painting process. The application allows users to overlay customizable grids on reference images while providing control over grid density, opacity, and color. Project data is stored locally using SQLite, enabling artists to save, revisit, and manage their work across multiple sessions. In addition to grid customization, the application includes a canvas recommendation system that analyzes image dimensions and compares aspect ratios against a predefined collection of standard canvas sizes to identify the closest match. CanvaGrid consolidates project management, grid-based image transfer, and canvas size recommendations into a unified workflow, making it a practical digital assistant for art educators, professional painters, and hobbyists alike.

Part 1: Purpose and Target Users

The preparation phase is crucial for any traditional visual artwork, as it establishes structural proportions and composition boundaries. Part 1 introduces the purpose of CanvaGrid and the users it aims to support. By combining traditional artistic techniques with mobile technology, the application helps artists transfer reference images to physical canvases accurately and efficiently.

1.1 Purpose of the Application

The primary objective of CanvaGrid is to eliminate the tedious manual calculations and potential errors associated with measuring and scaling reference photos. In traditional fine arts, expanding a small reference picture onto a large canvas requires precise aspect ratio calculations so that the final drawing does not suffer from distortion or forced cropping.

CanvaGrid automates this entire pre-drawing phase. By providing real-time grid customization tools, it allows artists to break down complex compositions into smaller, manageable segments, helping them focus on one block at a time to ensure anatomical and perspective accuracy. Furthermore, through its smart canvas suggestion feature, the application analyzes the uploaded image to recommend the ideal physical canvas size and the exact grid lines the artist needs to mark down before starting their work.

1.2 Target Users

The application is tailored for a diverse group of users across art and education. It directly addresses the needs of traditional painters and sketch artists working with oil, acrylic, watercolor, or charcoal who regularly transfer photographic references onto physical canvases, wood panels, or drawing paper. Additionally, it serves art students and educators who utilize the grid method as a fundamental learning tool to train the eye in recognizing shapes, values, and negative space. Finally, the platform accommodates casual hobbyists and craft enthusiasts who want a structured, automated guide to help them achieve accurately proportioned results without needing advanced drafting training.

Part 2: Innovation and Differentiation

The mobile application market already features various baseline grid overlays (like Drawing Grid Maker, Artist's Grid Utility, etc.), however, they consistently suffer from significant functional gaps. Existing applications act merely as passive visual filters, treating the grid as a rigid, static graphic element. They often ignore the realities of the physical art world, forcing artists to perform complex manual divisions to map digital blocks to their physical canvases, which frequently results in skewed proportions or forced image cropping.

CanvaGrid addresses these limitations through several key features, starting with its canvas size recommendation system. Instead of forcing the user into a trial-and-error cycle, the application includes an aspect-ratio matching algorithm. When an image is uploaded, the app analyzes the image's ratio against a predefined collection of standard commercial canvas sizes, such as 40×40 cm, 50×70 cm, and 60×80 cm. It automatically identifies the closest commercial match and recommends the exact physical proportions, preventing image distortion or awkward borders.

Another major differentiation is the real-time grid customization. The editor workspace provides granular control over the grid structure via a live-updating user interface. Users can adjust grid density from 2×2 to 12×12 cells, control alpha-transparency ranges via sliders to prevent gridlines from obscuring fine image details, and hot-swap color values across high-contrast options, including Red, Green, Blue, White, and Black, to contrast perfectly against any dark or light image background.

Furthermore, CanvaGrid incorporates a persistent project management system. Unlike alternative tools that do not save data between sessions, the application includes a dedicated SQLite database storage infrastructure. This allows creators to permanently save multiple ongoing projects, securely caching the underlying image paths alongside exact grid configurations, opacity levels, and color indices. Users can seamlessly revisit, rename, preview, or delete their workspace records, allowing artists to manage long-term projects across multiple sessions that match the extended timelines of traditional painting.

Finally, the application focuses on robust lifecycle state persistence. Instead of resetting or losing user progress when the screen layout changes, CanvaGrid implements strict state caching. Active project information, interface parameters, and open dialogs (such as Rename, Delete, or Help modals) are dynamically persisted using lifecycle callbacks and restored seamlessly. This ensures an uninterrupted user experience during configuration changes like screen rotations.

Part 3: User Scenarios and Functionality

The core architecture of CanvaGrid translates into concrete user interactions through structured workflows. Part 3 illustrates how different artist profiles leverage the application to solve real-world drafting challenges, demonstrating the practical transitions between image analysis, grid customization, and data persistence. These workflows emphasize the application's utility, showing how the interface design accommodates the physical and temporal demands of traditional art creation.

3.1 User Scenario I - The Painter

A traditional landscape painter wants to replicate a photographic reference onto a canvas but struggles to determine the correct proportions and setup. The artist launches the application and opens the canvas size recommendation tool, which prompts for access to the device's image gallery. Upon selecting a high-resolution landscape photo, the application analyzes the file's dimensions and extracts its aspect ratio efficiently. The app matches the photo's proportions against its predefined database of commercial canvas sizes, displaying the ideal physical dimensions in centimeters along with a recommended grid density. Satisfied with the match, the artist taps the confirmation button, which transfers the photo directly into the main editing workspace with the recommended configuration automatically applied.

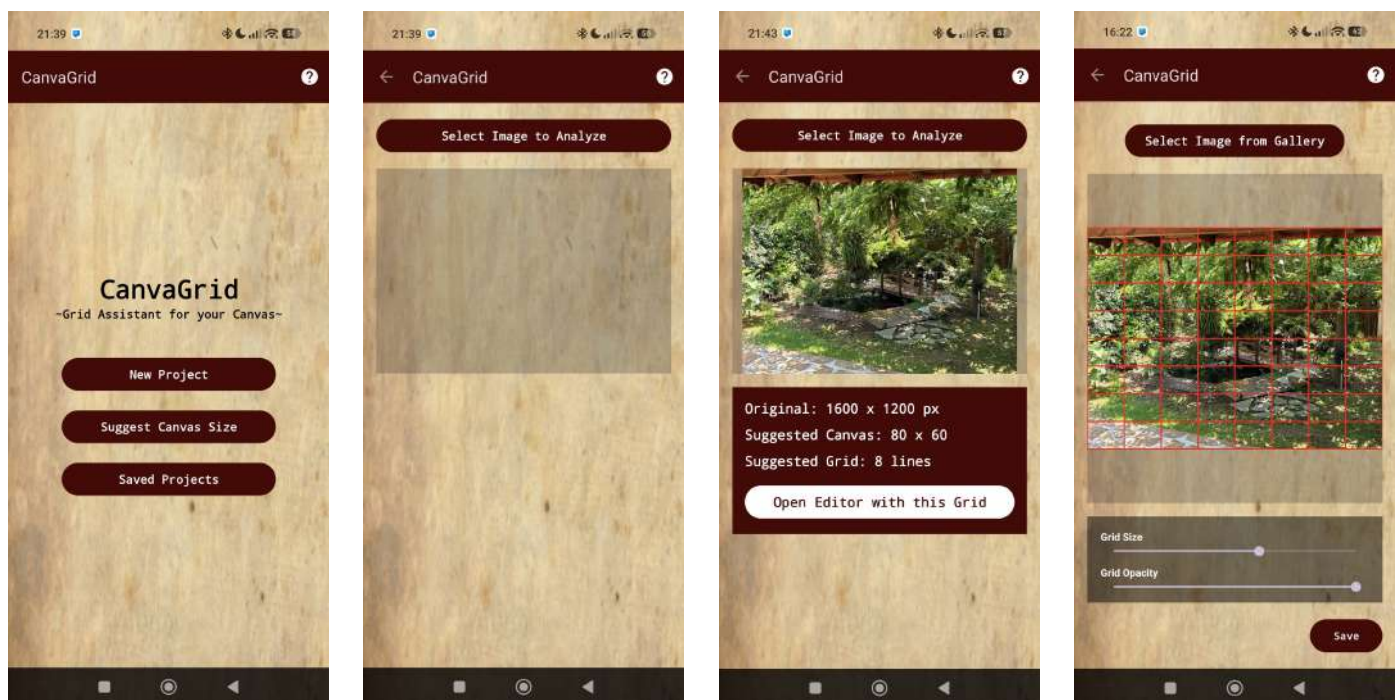


Figure 3.1.1: User presses "Suggest Canvas Size" button.

Figure 3.1.2: User presses "Select Image to Analyze" button, and chooses image from device.

Figure 3.1.3: After image selection, the user can see the canvas and grid suggestion.

Figure 3.1.4: User presses "Open Editor with this Grid" button, and the suggested grid settings are set on the selected image.

3.2 User Scenario II - The Art Student

An art student preparing a complex portrait composition needs to customize a grid overlay to contrast against an image with heavy shadows and high-contrast lighting. After loading the image into the editor workspace, the student notices that standard dark grid lines disappear in the darker regions of the photo. Using the editor's control panel, the student instantly changes the color of the lines to a high-contrast vibrant green. Furthermore, the user adjusts the alpha-transparency seek bar to lower the opacity of the grid, ensuring the guidelines remain visible without obscuring the fine anatomical details of the subject's face. The student then increases the grid density, triggering a real-time layout refresh that provides the precise guidelines needed for an accurate charcoal transfer.



Figure 3.2.1: User presses “New Project” button.



Figure 3.2.2: User presses “Select Image from Gallery” button, and chooses image from device.



Figure 3.2.3: After image selection, the user cannot see the grid lines due to intense shadows.

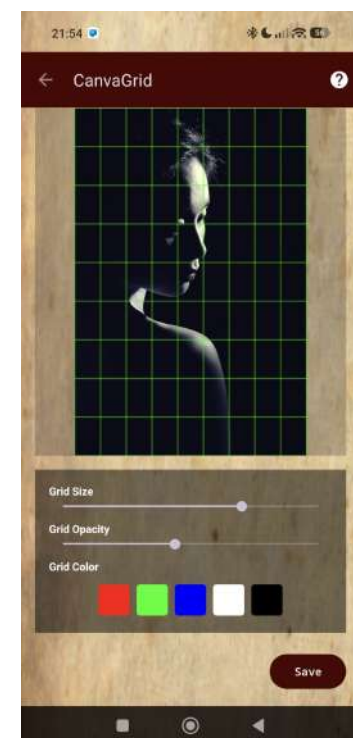
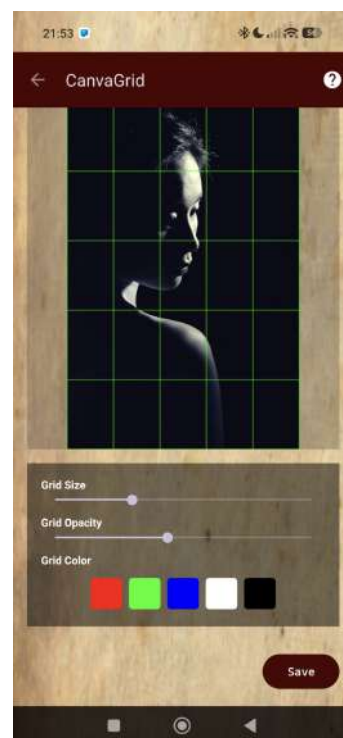
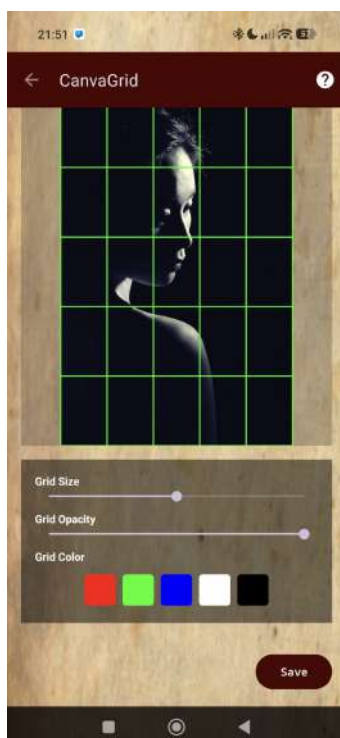


Figure 3.2.4: User changes line color by choosing the green option of "Grid Color".

Figure 3.2.5: User changes line opacity by sliding the "Grid Opacity" bar.

Figure 3.2.6: User changes line density by sliding the "Grid Size" bar.

3.3 User Scenario III - The Hobbyist

A hobbyist opens the application to update their records after finally completing a multi-week painting project. The artist navigates to the saved projects screen, opens the item options menu, and selects "Simple View" to open a full-screen preview and verify it is the correct file. After confirming, the user closes the preview, re-opens the menu, and selects the rename option to change the title to "Complete." While typing, the user accidentally rotates the device, causing a screen layout shift. However, thanks to the application's lifecycle state preservation, the rename dialog remains open and the already typed text is securely restored, allowing the hobbyist to seamlessly finish the renaming without losing progress.



Figure 3.3.1: User presses "Saved Project" button.



Figure 3.3.2: User searches for the project they want to edit.

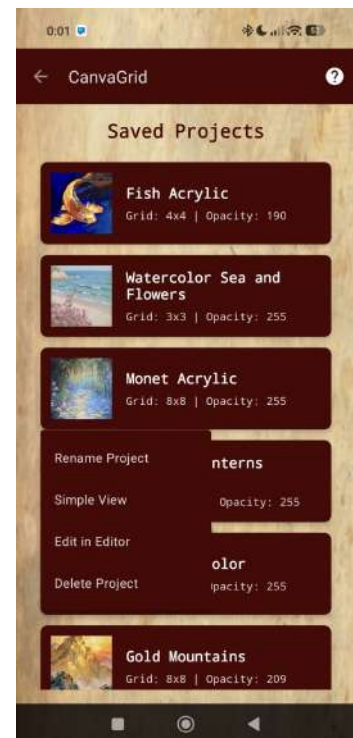


Figure 3.3.3: User presses the project they want to view.



Figure 3.3.4: User selects the "Simple View" option for simple project preview.



Figure 3.3.5: User selects the "Rename Project" option to change the project's name.

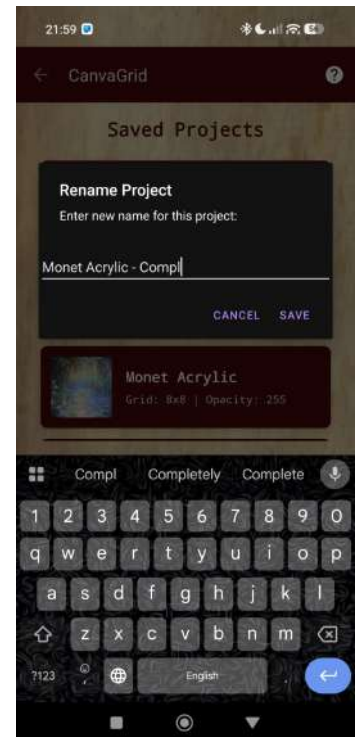


Figure 3.3.6: User starts typing the new project name.

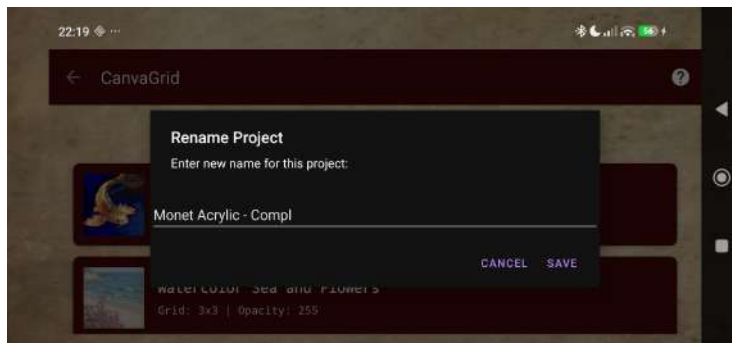


Figure 3.3.7: User accidentally rotates device, but their progress is saved.



Figure 3.3.8: User successfully renames the selected project.

Part 4: Source Code Explanation

The technical implementation of CanvaGrid follows a modular architecture that separates data persistence, user navigation, image processing, and graphical rendering into dedicated components. The local storage layer is built around the **ProjectContract** and **ProjectDbHelper** classes, which manage project information through an SQLite database and convert stored records into lightweight **ProjectModel** objects for use throughout the application. Navigation between the application's workflows is coordinated through the activity structure, with **MainActivity** serving as the central entry point and routing users to the editor, canvas recommendation tool, or saved project manager. Saved projects are displayed through **SavedProjectsActivity** and **ProjectAdapter**, while **PreviewFragment** provides a simplified read-only visualization of previously saved work. The main editing functionality is implemented within **EditorActivity**, where user interactions are connected to a custom **GridView** that renders grid overlays directly on top of selected images using Android transformation matrix calculations. Canvas recommendations are generated by **SuggestSizeActivity**, which analyzes image dimensions and compares aspect ratios against a predefined collection of standard canvas sizes to identify the closest match. Finally, user guidance and interface continuity are supported through the **HelpItem** and **HelpAdapter** components and lifecycle state restoration mechanisms. These mechanisms preserve open dialogs and user inputs during configuration changes like screen rotations .

4.1 Database Design and Architecture

The local storage subsystem of CanvaGrid is built around a reliable database structure that handles all aspects of project saving and management. This data layer consists of four specific Java components: the classes **ProjectContract**, **ProjectDbHelper**, **ProjectModel**, and the inner *ProjectEntry* static class. The schema definitions are securely stored within **ProjectContract** and its inner *ProjectEntry* class, which defines the exact blueprint of the database table. This file contains the names of the columns that the application needs to track, including the unique project ID, the user-defined project title, the storage path (URI) of the selected gallery image, the number of horizontal and vertical grid lines, the alpha opacity level of the grid, and the index representing the selected line color. The physical lifecycle of this database is managed by the **ProjectDbHelper** class, which extends Android's native *SQLiteOpenHelper*. Within its *onCreate* method, the helper executes the SQL command to create the projects table based on the contract configurations, while its *onUpgrade* method safely handles table resets during app updates. To transfer these records smoothly between the database and the user interface, the app uses the **ProjectModel** class as a simple data object. This class wraps all the project parameters into private variables and provides standard constructor and getter methods. When the app saves a new project or updates an existing one, it converts the properties of the **ProjectModel** into a standard *ContentValues* pair collection for safe database insertion. Conversely, when loading saved artwork, the database queries return a standard *Cursor* object, which is read sequentially in a loop to rebuild the list of active **ProjectModel** items before the data streams are closed to optimize device memory.

4.2 Main Hub Navigation and Saved Projects Management

The **MainActivity** class serves as the central navigation hub of CanvaGrid and manages access to the application's primary workflows. Inside its initialization phase, this class connects execution listeners to three main interactive buttons to establish separate operational pathways. The first path triggers an explicit intent that safely routes the user directly to a fresh editing workspace, managed by the **EditorActivity** class. The second path manages the canvas optimization workflow, directing the user task stack to the sizing assistant tool, managed by the **SuggestSizeActivity** class. The third path handles access to historical records (**SavedProjectsActivity** class), but introduces a crucial software validation safeguard: before initiating an intent to open the archives, the code invokes a specialized database check named *hasSavedProjects* on the local database persistence helper. If the query returns false, the activity intercepts the transition and flashes a user-friendly toast notification, preventing navigation into an empty screen and improving the overall user experience. Additionally, the class manages its application exit confirmations and user guide overlays using an internal tracking string variable that captures the active dialogue state. This metadata flag is automatically committed to the system bundle during destruction and re-evaluated inside the early initialization phase, allowing the activity to seamlessly reconstruct open alerts or scrollable tutorial views across device orientation changes.

The management and structural maintenance of the saved user archives are orchestrated by the **SavedProjectsActivity** class, which serves as the activity responsible for displaying and managing saved projects. This class handles the saved projects interface and display states of the portfolio window, directly implementing the callback structures defined within the custom *ProjectAdapter.OnProjectActionListener* contract. Instead of loading database rows during the initial creation phase, the activity uses the native *onResume* lifecycle callback to query the local SQLite helper. This design pattern guarantees that whenever the screen gains active user focus, it fetches an updated array of project data models, smoothly terminating its own context with an automated interface notification if the portfolio empties out. The class introduces a specialized navigation dispatcher that overrides traditional hardware back actions to support a multi-layered interface. If the system detects that a **PreviewFragment** overlay is currently stacked on top of the layout window, the back-press callback cleanly pops the fragment layer out of view to reveal the list. When the user is at the root directory of the portfolio, the activity implements a double-back-press mechanism to prevent accidental exits. A single press warns the user, while a rapid second press safely terminates the activity focus to prevent accidental app navigation. Furthermore, the class encapsulates the core business logic for dataset modification through dedicated execution structures. When row interactions register a modification request, the class handles the setup of parameterized alert prompts to manage project deletion queries or inline text renaming workflows. Upon successful execution, the class writes these updates directly to the persistence layer, updates its underlying data array index values, and notifies the adapter to refresh the affected list item and update the active view layout instantly. To seamlessly preserve these uncommitted text modifications during hardware screen rotations within this renaming workflow, a synchronous *TextWatcher* is embedded directly within the *showRenameDialog* method. The text is updated in the activity's global parameter,

activeProjectName, whenever the user types. When a physical orientation switch triggers an activity recreation cycle, this active text buffer is securely committed into the outgoing *onSaveInstanceState* bundle. Upon reconstruction, the initialization logic evaluates the saved data, automatically re-inflates the dialog, and repopulates the input layer with the exact cached string fragment, ensuring complete data persistence and an uninterrupted user experience.

The bridge between the database collection and the portfolio user interface is managed by the **ProjectAdapter** class, which extends the standard *RecyclerView.Adapter* pattern and utilizes an optimized internal *ProjectViewHolder* to prevent unnecessary structural layout recreation. Beyond inflating individual list items and formatting descriptive project parameters, this component encapsulates the primary interaction routing logic for the saved archive dashboard. Within its *onBindViewHolder* implementation, the class safely converts image path strings into functional assets, incorporating robust exception blocks that automatically fallback to standard system gallery icons to protect the layout sequence from rendering crashes. The main functionality of the adapter is driven by a contextual *PopupMenu* initialized directly upon user item selections. This menu bridges data operations and layout orchestration across four specialized control branches. For the "Simple View" routine, the class extracts the model attributes, packages them into a system *Bundle*, instantiates the **PreviewFragment** class, and executes an explicit fragment transaction using a smooth fade transition. For the "Edit in Editor" execution path, the adapter instantiates an explicit navigation intent, embedding all specific grid configuration values as intent extra primitives before shifting the application focus to the editor terminal. Conversely, for data modification and structural deletion sequences, the class relies on strict encapsulation practices; it forwards the target record parameters and active position indices back to the hosting container by utilizing the *OnProjectActionListener* interface callback structures, completely isolating visual representation from persistent storage mutations.

The **PreviewFragment** class provides a lightweight read-only preview of saved projects. This structural component is engineered to deliver a clean view of the artwork, completely stripped of editing controls and sliders. The lifecycle logic within its *onCreate* callback implements a dual validation routine to guarantee data continuity across application states. When initialized for the first time, the class extracts the canvas parameters from the incoming arguments package generated by the list adapter. Conversely, if a screen rotation occurs while the preview is active, the class bypasses the initial arguments and directly rebuilds its instance fields using key tokens cached inside the persistent *savedInstanceState* bundle. The primary graphic synchronization takes place within the *onCreateView* lifecycle phase, where the class binds the image database paths to the display and introduces a critical rendering safeguard. Because mobile user interfaces require a brief processing window to determine the exact geometric dimensions of a displayed image, the fragment avoids running immediate drawing calculations, which could lead to skewed lines or visual alignment bugs. Instead, it schedules an asynchronous execution block by invoking the native *ivFull.post* runtime loop. This setup forces the layout queue to wait until the underlying image asset is fully measured and drawn on the screen viewport. Once this rendering pass completes, the code securely invokes configuration bindings on the nested **GridView**, passing the required line counts, transparency

values, and high-contrast colors before explicitly forcing a view invalidation pass to render the finalized grid cleanly over the graphic frame.

4.3 EditorActivity Workspace and Grid Rendering

4.3.1 EditorActivity Workspace Management

The core editing environment and canvas state are managed by the **EditorActivity** class, which serves as the central processing workspace for grid manipulation. Within its primary initialization block, this class configures interactive event listeners across multiple control panels to bridge user input with dynamic graphic updates. For line structure adjustments, it binds custom *OnSeekBarChangeListener* contracts to the density and opacity sliders, instantly feeding progression steps directly into the underlying grid layer. Image acquisition is handled by registering an asynchronous *galleryLauncher* routine through native activity result contracts. Upon file selection, the class extracts the content database pathway and executes a critical security command, *takePersistableUriPermission*, which binds long-term read tokens to the image resource so the application can access the asset across subsequent device boot cycles. To safeguard active progress against abrupt environment changes, the class implements comprehensive layout lifecycle state preservation. Inside its state-saving sequence, it caches active configuration primitives alongside the current workspace project identifier and active dialog string tokens. During runtime restoration phases, the code reads these cached bundles, opens a safe input stream to decode the gallery asset back into an active memory bitmap, and programmatically evaluates the dialog string flags to automatically rebuild open warning dialogs or save options alerts without disrupting the artist's focus.

The backend utility of the workspace features an image export functionality configured within the *saveEditorImageWithGrid* method. When an artist requests a hard copy export, this method intercepts the current viewport state, extracts the underlying data array bitmap, and instantiates a high-fidelity mutable copy using the asset's native configurations. It then wraps this bitmap inside an active Android *Canvas* element and configures a structural *Paint* graphics object, transferring the selected color indices, layout stroke widths, and alpha opacity channels directly into the rendering loop. By dividing the bitmap's physical pixel width and height by the targeted row constraints, the code runs a loop that draws permanent vector lines directly across the pixel matrix. Once the drawing pass is completed, the system organizes a collection of file metadata parameters inside an Android *ContentValues* mapping, defining explicit title strings and relative storage paths. It then safely stream-compresses the finalized grid asset directly into the system's external picture directory via the native *MediaStore* content resolver, providing safe resource export execution that avoids application memory overload.

4.3.2 GridView Overlay Rendering

The actual visual rendering of the grid system on the device screen is executed by the custom **GridView** class, which extends the native Android *View* component to function as an advanced, custom drawing layer. Rather than stretching arbitrary lines across the entire window layout, this

component is structurally engineered to anchor its vector drawings directly to the real pixel boundaries of the underlying artwork. During its *onDraw* lifecycle phase, the class searches the parent view hierarchy to resolve references: if a target image element was not manually linked via the *bindToImageView* dispatcher, the code programmatically scans the parent view hierarchy to dynamically resolve references for either the editing workspace image view or the preview component frame. If no background asset is detected, the drawing sequence immediately short-circuits to preserve system rendering cycles.

Once an active graphic is verified, the main rendering logic uses low-level matrix transformations to isolate the exact geometric footprint of the photo. By extracting a float array from the image's native transformation matrix, the code multiplies the asset's intrinsic dimensions by the active scaling fields, *MSCALE_X* and *MSCALE_Y*. This matrix evaluation yields the exact on-screen width and height of the rendered image, allowing the view to mathematically compute the coordinate offsets for the four primary boundary corners. After configuring the painter's structural alpha transparency channel, the class executes two automated mathematical loops. These loops calculate sequential incremental offsets to draw inner horizontal rows and vertical columns that scale smoothly with the user's slider positions. Finally, the rendering sequence invokes a precise *drawRect* command using the calculated boundary offsets, enclosing the photo in a clean structural outer frame and ensuring the visual assistance lines update immediately whenever a modifier triggers an interface invalidation pass.

4.4 SuggestSizeActivity and Canvas Recommendation Algorithm

The automatic calculation of proportional canvas dimensions and corresponding grid structures is controlled by the **SuggestSizeActivity** class, which serves as the canvas recommendation component of the application. The operational workflow is driven by an image analysis pipeline contained within the private *analyzeImage* method. To safeguard the host mobile device from memory exhaustion when handling high-resolution camera images, this method opens a temporary data stream and initializes a specialized *BitmapFactory.Options* configuration wrapper. By explicitly enabling the *inJustDecodeBounds* flag asset, the Java runtime intercepts the metadata header of the file to extract its native pixel width and height without allocating heap memory blocks for the actual pixel arrays. Once the dimensions are resolved, the engine calculates the aspect ratio of the photo using standard floating-point division.

The recommendation algorithm compares this calculated aspect ratio against an embedded predefined canvas size table named *PREDEFINED_SIZES*, which stores commercial standard art canvas dimensions across square, portrait, and landscape form factors. The algorithm calculates the absolute difference between aspect ratios and selects the closest match. It iterates through the matrix configurations, computes the mathematical delta between the image ratio and each canvas template ratio using absolute difference checks, and captures the array index that yields the absolute minimum variance. After isolating the closest physical size match, the class normalizes the target cell increments to fit within predefined bounds and binds the calibrated physical size measurements directly to the active user interface panel. Upon user validation, the class requests explicit persistable read permissions for the content database path, packs the calculated optimal grid values into an

explicit navigation intent extra, and securely forwards the structured environment state directly into the workspace terminal while calling *finish()* to close the activity and free system resources.

4.5 Help System and Dialog Lifecycle Management

To maintain an intuitive user experience without cluttered layouts, CanvaGrid isolates its onboarding instructions and interface alerts into a unified, modular dialog management subsystem. The architectural backbone of the contextual help components relies on the **HelpItem** class, a clean data model designed to encapsulate individual instruction states. This immutable class stores a trio of primitive and resource references: a string header for the task title, a secondary text block for the operational description, and an integer identifier pointing directly to native vector drawable assets. By modeling help sequences into structured array lists of these objects, the application decouples instructional copy from the core logic of the activities.

The structural rendering and view population of these instructional models inside the dialog overlays are managed by the **HelpAdapter** class, which extends the standard *RecyclerView.Adapter* pattern and utilizes an internal static *HelpViewHolder* to cache widget bindings efficiently. Inside its *onBindViewHolder* phase, the adapter maps the structured data fields directly onto the text layouts, incorporating a fallback check; if the layout engine resolves an unassigned drawable integer identifier, it automatically bypasses the invalid target and binds a native system assistance drawable instead, protecting the overlay view sequence from visual crashes.

Beyond plain data mapping, the application implements a strict lifecycle state monitoring pattern across all primary workspace screens to prevent state loss during screen rotation instances. Dialog instances, such as application termination prompts, image replacement warnings, and database overwrite choices, are tracked using a centralized primitive string state marker named *activeDialogType*. When a configuration change occurs (such as switching from portrait to landscape orientation), the active hosting activity intercepts the destruction phase and caches this string token, alongside target entity primary IDs or active list position markers, directly into the outgoing *onSaveInstanceState* bundle. During the subsequent recreation pass, the initialization block evaluates these active state keys; if a valid identifier is resolved, the system immediately invokes the corresponding builder subroutine to re-inflate and re-anchor the exact alert or scrollable help modal that was visible prior to the layout shift, ensuring that dialogs remain visible after configuration changes.

4.6 Scoped Storage Architecture and Data Privacy Management

CanvaGrid follows the modern Android Scoped Storage model and relies on the Storage Access Framework (SAF) for image selection and file access. Instead of requesting broad storage permissions, the application delegates file selection to the system file picker, allowing users to explicitly choose the images they wish to use. This approach improves privacy by restricting access only to the files selected by the user. When an image is imported through either **SuggestSizeActivity** or **EditorActivity**, the application obtains a content URI and requests persistable read permissions

using the Android framework. These permissions allow CanvaGrid to continue accessing previously selected images across application restarts and device reboots without requiring the user to reselect the files. This design simplifies permission management while ensuring compatibility with current Android storage policies. It also supports the project's persistence features, as saved projects can safely reference previously selected images stored outside the application's private directory without causing permission-related failures when projects are reopened.

Part 5: Future Additions

5.1 Potential AI Integration and Tablet Adaptations

A compelling future direction for CanvaGrid could explore the integration of an on-device Vision-Language Model (VLM). Instead of relying only on basic aspect ratio calculations, this AI feature could analyze the actual content and complexity of an uploaded image. For instance, detailed or multi-figure compositions usually need larger canvases to preserve fine details, while simpler minimalist designs look great on smaller surfaces. Based on this analysis, the app could recommend the ideal canvas size and grid setup dynamically. Concurrently, to optimize the application for a broader audience, including older hobbyists, the user interface could be adapted for tablets. Instead of just stretching the current phone layout, this upgrade would introduce a multi-pane fragment design. This would allow the editor workspace and the configuration sliders to sit side-by-side on a single screen, reducing navigation steps and making the app much easier to use.

5.2 Future AR Features and Cloud Syncing

Beyond standard image processing, the core application could expand into interactive environments by adding an Augmented Reality (AR) layer. Using the device's camera and standard computer vision libraries, the app could automatically detect a physical blank canvas in the room. Once the canvas boundaries are identified, the app could project the customized grid layout directly onto the physical canvas surface through the camera view, helping artists sketch their outlines more accurately. Finally, to allow users to access their projects from multiple devices, the local SQLite database could be connected to a cloud synchronization backup. The application would automatically mirror saved project records and image paths to a secure cloud database. This would support safe data recovery and cross-device syncing without slowing down or changing the core efficiency of the local database structure.